



THE NATIONAL CERT OF **PAKISTAN**

L-Block, Pak Secretariat, Islamabad



Submitted By	Hafiz Muhammad Moawiz
Degree	BS - CY
Domain	Software Security Testing
Submitted To	Team NCERT

Manual Penetration Testing — OWASP Top 10

Week 2 Task Report (26-7-2026)

#	Target URL	Target Name	Status
1	http://localhost:3000	OWASP Juice Shop	Completed

1. Executive Summary

I did manual security testing on the OWASP Juice Shop web application. This app is used for practice and it has vulnerabilities on purpose. I tested it like a real web application, and I checked for common OWASP Top 10 issues.

I found 6 issues in total. Out of these, 2 are Critical, 3 are High, and 1 test (CSRF) came out safe, meaning the application was protected for that specific check. Every finding in this report has proof (screenshot) showing that I actually did the attack and it worked.

The most serious problems I found are SQL Injection in the login form, which let me log in as admin without a real password, and a Cross-Site Scripting (XSS) bug that let me steal a user's session cookie. Both of these are very dangerous because an attacker can take over accounts using them.

1.1 Summary Table

#	Finding	Severity	Status
1	SQL Injection — Login Bypass	Critical	Confirmed
2	XSS (DOM-based) — Search Box	High	Confirmed
3	XSS — Session Cookie Theft (PoC)	Critical	Confirmed
4	CSRF — Tested, Confirmed Safe	Safe	Not Vulnerable
5	Broken Authentication — Weak Password / No Rate Limit	High	Confirmed
6	Security Misconfiguration — Exposed /ftp Directory	High	Confirmed

2. Scope and Methodology

Target: OWASP Juice Shop (intentionally vulnerable practice web application)

Testing Type: Manual Black-box Testing

Tools Used: Burp Suite Community Edition

I checked the application manually for common OWASP Top 10 problems. This included:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)
- Broken Authentication
- Security Misconfiguration
- Missing Security Headers

For every bug I found, I took a screenshot to show proof. This means whatever attack or script I wrote; I also show the result (popup, login, or data) that proves it actually worked.

Finding 1: SQL Injection — Authentication Bypass

Severity: Critical **CVSS-style Risk:**

What I did

I checked the login page of Juice Shop to see if it is safe from SQL Injection. I put a SQL Injection payload in the email field instead of a normal email. The application did not clean or check my input properly, so my payload went straight into the database query.

Steps I followed

- I opened the Juice Shop login page.
- In the email field, I typed: ' OR 1=1--
- I left the password field empty (or put anything, it did not matter).
- I clicked Login.

Proof of Concept (Evidence)

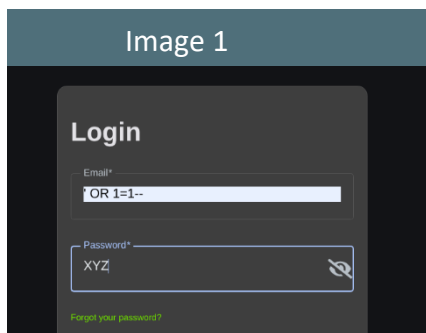
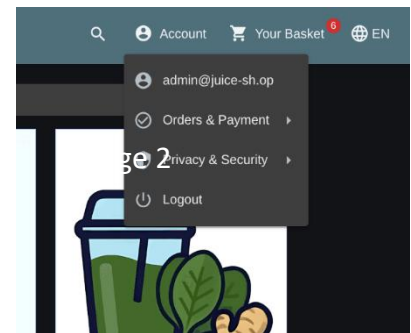


Image 1 — SQL Injection payload ' OR 1=1-- entered in the email field of the login form.

Image 2 — Login was successful and I got logged in as admin@juice-sh.op without knowing the real password. This confirms the SQL Injection worked.



Risk / Impact

This is a very serious problem. Because of this bug, anyone can log in as the admin user without knowing the real password. An attacker can take full control of the admin account and see or change data of other users. This can lead to complete account takeover.

Remediation Recommendation

- Use parameterized queries / prepared statements instead of building SQL queries directly from user input.
- Validate and sanitize all input coming from the login form.
- Use an ORM (Object Relational Mapper) which handles queries safely.
- Do not show detailed database errors to the user.

Finding 2: Cross-Site Scripting (XSS) — DOM-based, Search Box

Severity: **High** CVSS-style Risk:

What I did

I checked the search box of Juice Shop to see if it is safe from XSS. I put a script-based payload inside an iframe tag and searched for it. The application did not clean my input, so my script ran inside the browser.

Steps I followed

- I opened the Juice Shop search box.
- I typed this payload: `<iframe src="javascript:alert('xss')">`
- I pressed Enter to search.
- A popup (alert box) appeared, which means my script executed inside the page.

Proof of Concept (Evidence)

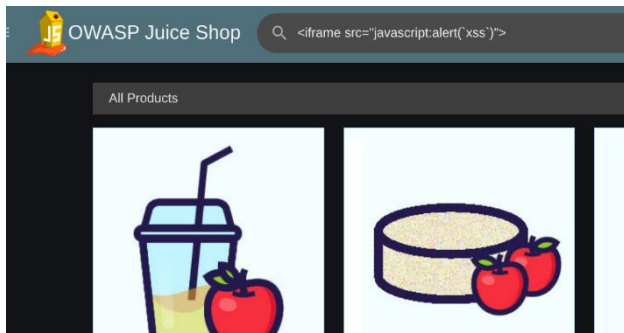


Image 3 — XSS payload `<iframe src="javascript:alert('xss')">` entered in the search box.

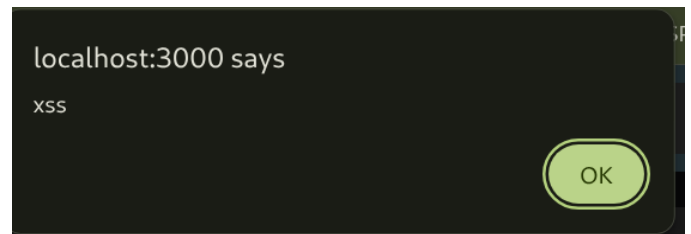


Image 4 — Alert popup fired, confirming the DOM-based XSS payload was executed by the browser.

Risk / Impact

Because the search box runs script code without checking it first, an attacker can send a malicious link to a victim. When the victim opens that link, the attacker's script runs inside the victim's browser. This can be used to steal cookies, redirect users to fake pages, or perform actions on behalf of the victim.

Remediation Recommendation

- Sanitize and encode all user input before showing it back on the page (output encoding).
- Use a strong Content Security Policy (CSP) header to stop inline scripts from running.
- Do not use dangerous sinks (like `innerHTML`) with unsanitized user input.
- Validate input on both client-side and server-side.

Finding 3: Cross-Site Scripting (XSS) — Session Cookie Theft (Proof of Concept)

Severity: Critical **CVSS-style Risk:**

What I did

After confirming XSS worked in the search box, I wanted to prove how dangerous it really is. So I wrote a script to read the browser's cookie (`document.cookie`) and show it in a popup. This proves that an attacker can steal the session cookie and JWT token of a logged-in user using the same XSS bug.

Steps I followed

- I used the same vulnerable search box.

- I wrote a payload using document.cookie to grab the cookie value and display it in an alert popup.
- I ran the payload in the search box.
- A popup appeared showing the session cookie and JWT token value, proving that cookie theft is possible.

Proof of Concept (Evidence)

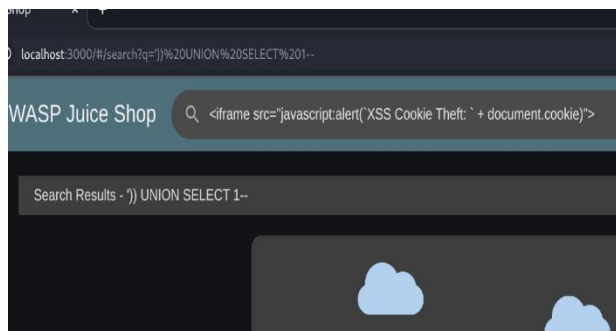


Image 8 — XSS payload written to steal the session cookie using document.cookie.

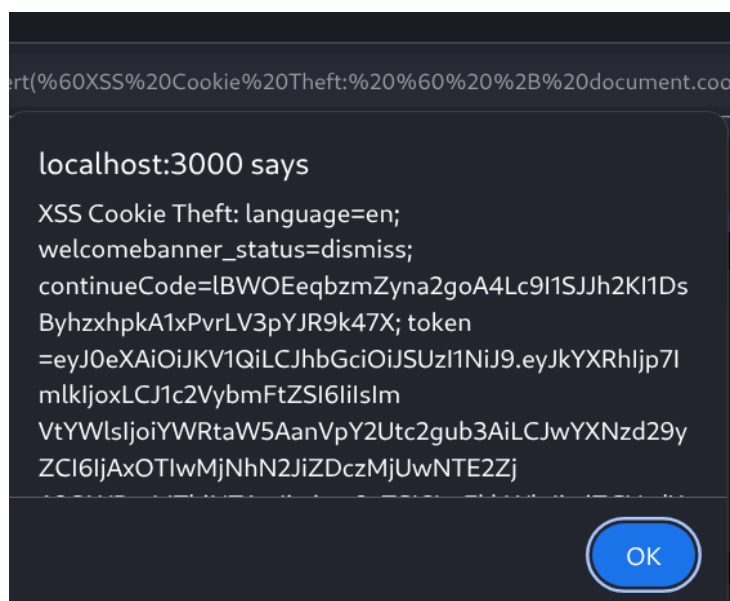


Image 9 — Popup showing the stolen session cookie and JWT token, proving that session hijacking is possible through this XSS bug.

Risk / Impact

This is a critical finding. If an attacker steals the session cookie or JWT token of a user, they can log in as that user without knowing their password. This is called session hijacking. An attacker could take over any logged-in user's account, including an admin, just by tricking them into opening a malicious link.

Remediation Recommendation

- Fix the root XSS issue (see Finding 2 remediation) — this will also remove the cookie theft risk.
- Mark session cookies as HttpOnly so JavaScript cannot read them at all.
- Mark cookies as Secure so they are only sent over HTTPS.
- Use short-lived tokens and refresh them often to reduce damage if one is stolen.

Finding 4: Cross-Site Request Forgery (CSRF) — Tested, Confirmed Safe

Severity: Safe **CVSS-style Risk:**

What I did

I tested if the application is vulnerable to CSRF on the address/profile update endpoint. I removed the Authorization header from the request and sent it again using only the cookie, to check if the server accepts requests without the proper token.

Steps I followed

- I intercepted a request to the address endpoint using Burp Suite.
- I removed the Authorization header from the request.
- I sent the request again using only the session cookie.
- The server replied with a 401 Unauthorized error, which means the request was rejected.

Proof of Concept (Evidence)

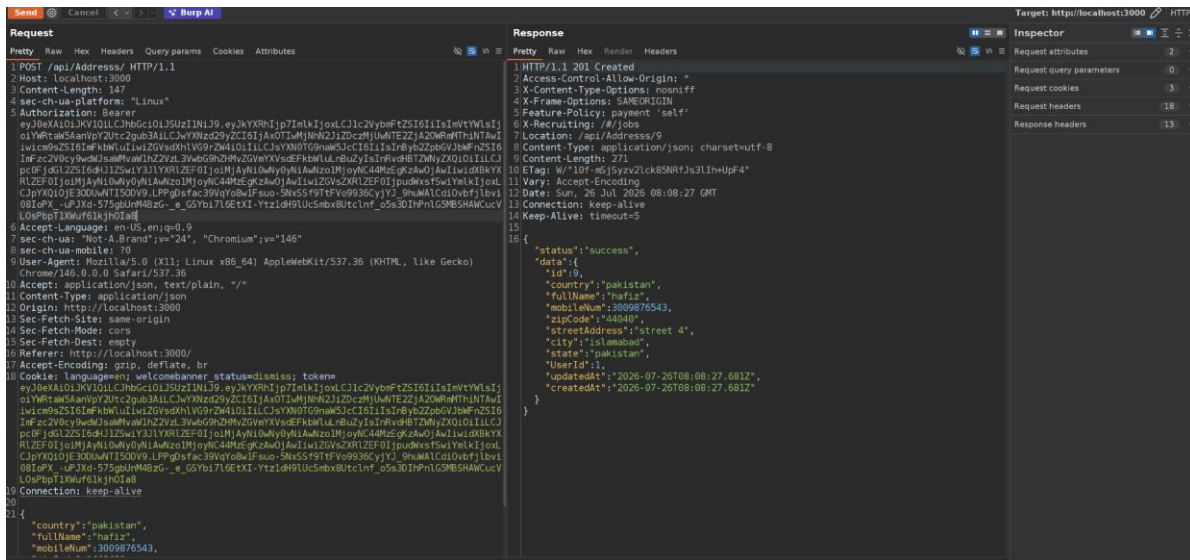


Image 5 — Authorization header removed from the request; server responded with 401 Unauthorized, showing the endpoint is protected by a Bearer token and not vulnerable to CSRF.

Risk / Impact

No risk found here. The application uses a Bearer token for authentication instead of only relying on cookies. This means a normal CSRF attack (which only uses the victim's cookies automatically) will not work here, because the attacker cannot guess or steal the Bearer token easily. This is a good security practice by the developers.

Remediation Recommendation

- No fix required for this specific test — the endpoint is already protected.
- It is still recommended to keep using Bearer tokens (not cookie-only auth) for all sensitive actions across the whole application.
- Continue to review other endpoints to confirm they follow the same protection.

Finding 5: Broken Authentication — Weak Password Accepted and No Rate Limiting

Severity: High **CVSS-style Risk:**

What I did

I checked how strong the login and password system is. I tried a very weak/common password on the admin account, and I also tried logging in many times in a row to see if the application blocks me after multiple failed attempts.

Steps I followed

- I tried the password 'admin123' on the admin account.
- The application accepted this weak password without asking for a stronger one.
- I then tried logging in with wrong passwords many times in a row.
- No lockout, no CAPTCHA, and no delay appeared after multiple failed attempts.

Proof of Concept (Evidence)

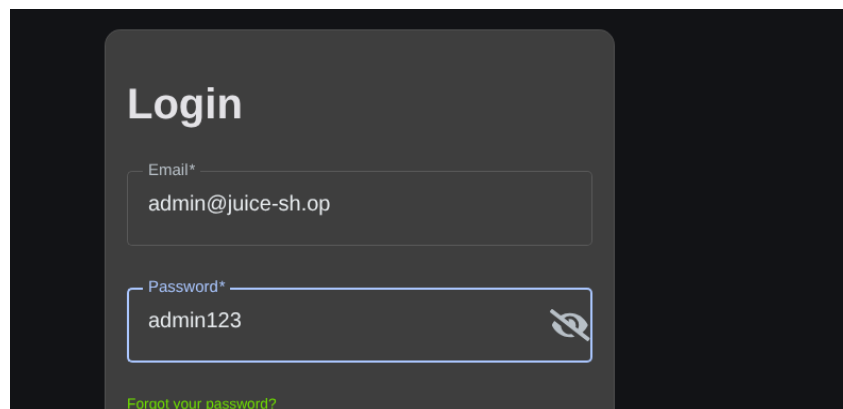


Image 6 — A weak password (admin123) was accepted by the application on the admin account, and repeated login attempts were not blocked.

Risk / Impact

Because weak passwords are allowed and there is no rate limiting or CAPTCHA, an attacker can easily guess passwords using a brute-force attack. Combined with a weak password, this makes it very easy for an attacker to break into user or admin accounts.

Remediation Recommendation

- Enforce a strong password policy (minimum length, mix of letters/numbers/symbols).
- Add account lockout after a certain number of failed login attempts.
- Add CAPTCHA after a few failed attempts to stop automated brute-force tools.
- Add rate limiting on the login endpoint (e.g., limit requests per IP per minute).

Finding 6: Security Misconfiguration — Exposed /ftp Directory

Severity: High **CVSS-style Risk:**

What I did

While exploring the application, I checked if there are any exposed folders that should not be public. I found that the /ftp directory is open to anyone, without login, and it has sensitive files listed inside it.

Steps I followed

- I browsed to the /ftp path on the Juice Shop domain.
- The folder opened without asking for any login or password.
- I saw a list of files, including files like incident-support.kdbx, acquisitions.md, and backup files.

Proof of Concept (Evidence)

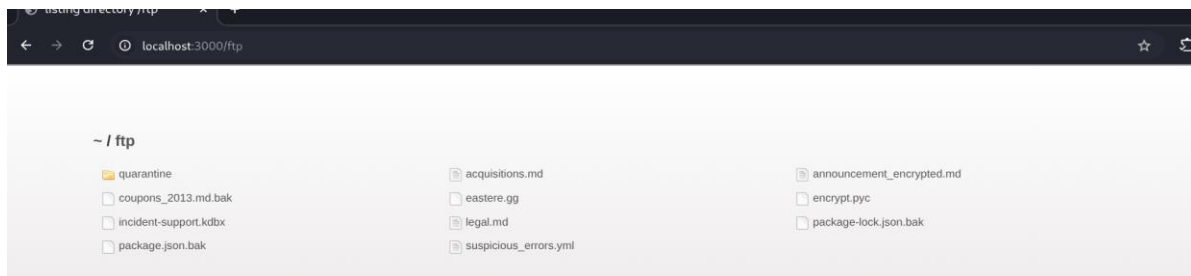


Image 7 — The /ftp directory is publicly accessible without authentication, showing sensitive files such as incident-support.kdbx, acquisitions.md, and backups.

Risk / Impact

This is a serious information disclosure problem. Sensitive files like password database files (.kdbx) and internal documents are visible to anyone on the internet without login. An attacker can download these files and try to crack passwords or read confidential company information.

Remediation Recommendation

- Remove directory listing on the web server configuration.
- Move sensitive files outside of the publicly accessible web folder.
- Add proper authentication for any folder that contains internal or sensitive files.
- Regularly check the server for files that should not be public.

The High findings (Search box XSS, weak password/no rate limit, and exposed /ftp folder) are also serious and can be used by an attacker to steal data or break into accounts, so they should be fixed soon after the Critical ones.

4. Conclusion

I did manual testing on OWASP Juice Shop and found 6 issues related to the OWASP Top 10. Every finding has proof through screenshots showing that the attack really worked, not just a theory. The application has serious problems like SQL Injection and XSS that can lead to full account takeover, so these need to be fixed first.